

G3 - Rossmann Store Sales

Members: Arya, Ethan, Fung, Gus, Kathir, Zenti.

Executive summary by Ethan, introduction by Zenti and data preprocessing by Ethan.

Exploratory data analysis conducted by Ethan, Gus and Arya and assisted by Fung, Kathir and Zenti on different variables' effects on sales. Predictive modelling was split into different models. Kathir and Zenti conducted multivariate linear regressions, Ethan and Gus conducted logistic regressions and Fung and Arya conducted decision trees, random forests and support vector machines. Conclusions by Ethan. All members contributed to both the construction of presentation slides and the writing of this report.

EXECUTIVE SUMMARY

The task. Managers at Rossmann are keen to use their data to analyse the importance of a range of factors on sales and predict the number of sales based on this data. Our task is to filter through and prepare the data, conduct exploratory data analysis to analyse variable correlations with sales and construct various models to predict the number of sales based on other variables.

Our approach. Each group member will use the same cleaned dataframes, but may apply new transformations to the dataframes depending on the requirements of their analysis. Members will be assigned different questions to investigate regarding exploratory data analysis, such as how competition distance affects the number of sales made by a particular store. The findings will then be brought together to help inform our strategy when it comes to predictive modelling. The predictive modelling will also be split up into different sections, this time based on the type of model we were attempting to fit to the data. All of multivariate linear regressions, logistic regressions, decision trees, random forests and support vector machines will be employed to try and produce informative models that can accurately predict the number of sales.

The vision. Once we find the most impactful variables on sales and the best models to predict the number of sales, Rossmann will be able to use this information to better plan for the future and maximise profits. By focusing on the most factors that influence sales the most and understanding how to change them to increase sales, Rossmann will gain a strategic advantage over competitors.

INTRODUCTION

Who is Rossmann? Dirk Rossmann GmbH, commonly referred to as Rossmann, is one of the largest drugstore chains in Europe. It has around 56,200 employees and more than 4000 stores. The company was founded in 1972 by Dirk Rossmann, with its headquarters in Burgwedel, Germany the Rossmann family own 60% of the company. The company has had an increase in revenue every year of the past decade and in 2023 it had an annual revenue of around 13.9 billion euros in Germany, Poland, Hungary, The Czech Republic, Turkey, Albania, Kosovo, Spain and Azerbaijan.



Figure 1.1.1, A Rossmann store.

The Rossmann dataset is a “pooled-repeated measures” dataset, whereby multiple observations from different stores are grouped together. Training data was provided for 1,115 Rossmann stores. This data was what is often used to analyse relationships between features of the stores and how they affect the success of the stores. This information is also used to construct

predictive models that would be then used to steer the Rossmann store management in the best direction in terms of maximising sales and revenue.

The data. The two main Data sets provided are the Train dataframe and the Store Dataframe. The “Train Dataframe” includes all historical data of the store, including sales (figure 1.2.1). These were day to day performances of the stores from the past.

	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	SchoolHoliday
Store								
1	5	2015-07-31	5263	555	1	1	0	1
2	5	2015-07-31	6064	625	1	1	0	1
3	5	2015-07-31	8314	821	1	1	0	1
4	5	2015-07-31	13995	1498	1	1	0	1
5	5	2015-07-31	4822	559	1	1	0	1

Figure 1.2.1, The train dataframe.

The data given in the Train dataframe includes the unique store ID for each store, the number of sales on the day, the date, which day of the week it was given by a number from 1 through 7, the number of customers on the day, whether or not the store was open on the day given by [1 =Yes, 0 = No], whether or not the store was running a promotion on the day, whether or not it was a school holiday on the day as well as whether or not it was a state holiday on the day given as [a = public holiday, b = Easter Holiday , c = Christmas Holiday, 0 = None].

As for the “Store dataframe” , it contains supplemental information about the stores. So it gives information about each unique store (figure 1.2.2).

	Store	StoreType	Assortment	CompetitionDistance	CompetitionOpenSinceMonth	CompetitionOpenSinceYear	Promo2
0	1	c	a	1270.0	9.0	2008.0	0
1	2	a	a	570.0	11.0	2007.0	1
2	3	a	a	14130.0	12.0	2006.0	1
3	4	c	c	620.0	9.0	2009.0	0
4	5	a	a	29910.0	4.0	2015.0	0
5	6	a	a	310.0	12.0	2013.0	0

Figure 1.2.2, The store dataframe.

The information given in this dataframe were the different store types, as in it differentiates the four different store models: a,b,c,d. The letters represent standard drug stores, convenience stores, flagship stores and specialty stores respectively. Other information included the store's assortment level [a = basic, b = extra, c = extended]. As well as the distance in metres to the nearest competitors, the times when the competitors opened up and the information even included whether or not the store was running a continuing Promotion [Promo2].

In this study, we analyse the intricate dynamics of Rossmann store's data to uncover valuable insights and trends that inform business strategies. Through comprehensive analysis and predictive modelling, we aim to enhance our understanding of the sales patterns and customers behaviours. By leveraging advanced statistical techniques and machine learning algorithms, we seek to develop robust models that not only forecast future sales but also facilitate data-driven decision making. Ultimately, our objective is to provide actionable recommendations that can optimise operations, improve marketing effectiveness, and drive overall growth for Rossmann stores.

DATA PREPROCESSING

Our data came in the form of two csv files: "Rossmann_store.csv" and "Rossmann_train.csv". We began by reading them into a jupyter notebook and converting them to a pandas dataframe to create two data frames that will be the basis of our analysis.

The train dataframe. We reordered the sales column in the train dataframe based on descending order of sales so that the data frame had some sensible order rather than being ordered based on a meaningless store number (figure 2.1.1). This also revealed that stores 909, 262 and 57 were likely some of the more successful Rossmann stores (since they each had a day with very high sales) and that the highest number of sales likely occurs during the year 2015. The state holiday column has categorical data labelled 0, a, b and c but this was only dealt with for analysis that required this column to be in a different format.

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	SchoolHoliday
0	909	1	2015-06-22	41551	1721	1	0	0	0
1	262	5	2015-04-03	38722	5132	1	1	b	0
2	262	5	2015-05-01	38484	5458	1	1	a	0
3	262	4	2015-05-14	38367	5192	1	0	a	1
4	57	1	2014-06-16	38037	1970	1	1	0	0
...	...	...	...	...	...	...	...	...	...
1017204	762	7	2014-10-26	0	0	0	0	0	0
1017205	761	7	2014-10-26	0	0	0	0	0	0
1017206	760	7	2014-10-26	0	0	0	0	0	0
1017207	759	7	2014-10-26	0	0	0	0	0	0
1017208	1115	2	2013-01-01	0	0	0	0	a	1

Figure 2.1.1, The prepared train dataframe

We used `train_df[["Sales","Customers"]].describe()` to explore the entries in both the sales and customers columns of the train dataframe (figure 2.1.2). Since there were no obviously incorrect values, no values were removed.

	Sales	Customers
count	1.017209e+06	1.017209e+06
mean	5.773819e+03	6.331459e+02
std	3.849926e+03	4.644117e+02
min	0.000000e+00	0.000000e+00
25%	3.727000e+03	4.050000e+02
50%	5.744000e+03	6.090000e+02
75%	7.856000e+03	8.370000e+02
max	4.155100e+04	7.388000e+03

Figure 2.1.2, Descriptive statistics of sales and customers in train dataframe.

The store dataframe. In order to view the effect of variables in the store dataframe on sales, we decided to create a new column representing the cumulative sales in the store dataframe. This

was done by calculating the total number of sales in the train dataframe across the entire time period for each store and aligning this data with the store column in the store dataframe to ensure each cumulative sales statistic was given to the correct store. We then sliced the store dataframe to keep only the top 200 stores from the 1115 total stores according to their cumulative sales in hopes of keeping only the most accurate and reliable data (figure 2.2.1). This was achieved using a combination of `df['Sales'].sum()` and `df.nlargest(200, "Cumilative_Sum")` and `pd.merge()`.

	Store	Sales	StoreType	Assortment
0	262	19516842.0	1	0
1	817	17057867.0	0	0
2	562	16927322.0	1	2
3	1114	16202585.0	0	2
4	251	14896870.0	0	2
...	...	...	...	...
195	1045	6654188.0	0	2
196	872	6649861.0	0	2
197	344	6649189.0	0	2
198	905	6645066.0	0	0
199	814	6628829.0	3	2

Figure 2.2.1, Store, sales, store type and assortment columns of the prepared store dataframe.

We also converted a, b, c and d values to 0, 1, 2 and 3 values respectively in the store type and assortment columns for easier analysis. This was done using `df['StoreType'].map({'a':0,'b':1,'c':2,'d':3})`.

EXPLORATORY DATA ANALYSIS

Competition Distance and Sales. We explored the relationship between distance to the nearest competing store and sales using a time series graph split based on competition distance (figure 3.1.1). “Very close” means within 100m of the nearest competing store, “within walking distance” is 100-2000m, “within driving distance” is 2000-10000m and “isolated” is above 10000m. This is just a rough definition of distances constructed to simply see whether there is a correlation between competition distance and sales. In order to create this figure, we used `pd.to_datetime()` to convert the date column to a suitable format and imputed days where a store had 0 sales with the mean number of sales for stores of that competition distance on that day.

This was done to achieve a figure that was easier to read whilst not disturbing the shape of the graph.

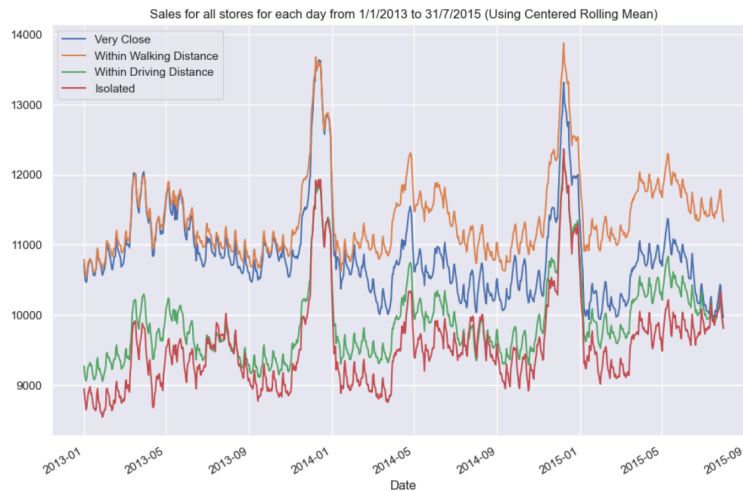


Figure 3.1.1, Sales over time based on competition distance.

Figure 3.1.1 clearly displays the seasonal nature of sales, specifically how sales peak during December. This can be explained by considering how Christmas and New Year both occur around December and so there will be far more customers who frequent the shopping centres where Rossmann stores may be situated. Also, stores which are very close or within walking distance to their nearest competing store clearly have more sales on average than stores which are within driving distance or isolated from their nearest competing store. One difference in the shape of sales over time between each of the four chosen distances from nearest competition is that stores which are very close to their nearest competing store actually have slowly decreasing sales over time. There are likely competing factors contributing to the results shown by this graph. On one hand, stores that are close to competition are going to have to share customers which clearly lowers sales. But being close to competition likely means the store is set up within a shopping district where there is a larger population of customers. The key takeaway is that a successful store will be set up in an area with high foot traffic, as long as the nearest competing store isn't extremely close by. This explains the success of stores which are within walking distance to their nearest competitor. Both time and competition distance seem to have an effect on sales and as such will be considered in our predictive modelling.

Promo and Sales. We explored the relationship between promotional events and sales using a time series graph and a box plot (figures 3.2.1 and 3.2.2). The time series graph displays sales over time, separated by promotional intensity: "High-Impact," "Moderate-Impact," and "Low-Impact." This categorization was created to observe how different levels of promotion influence sales trends over time. To create this figure, we used `pd.to_datetime()` to convert the date column into a suitable format, and imputed missing sales days with the average sales for that promotion level on the same day. This approach smooths the visual, making the trend clearer without distorting the data's shape.

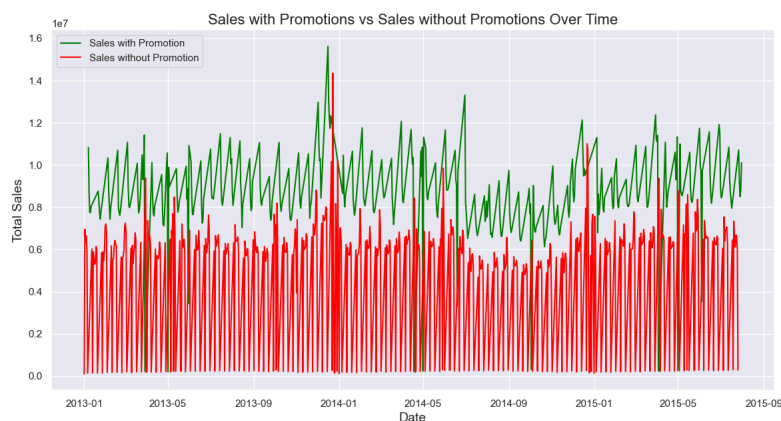


Figure 3.2.1, Time series graph of sales split based on promotions.

Figure 3.2.1 illustrates the seasonal nature of sales, with notable peaks during high-impact promotion periods, especially in December. This aligns with expected retail patterns, where holiday-driven promotions significantly boost sales. High-impact promotions yield the most pronounced spikes, while moderate-impact promotions provide a steady, smaller increase in sales. Low-impact promotions show minimal influence on sales trends, with nearly flat performance throughout the year. This demonstrates how promotional intensity and timing together drive seasonal sales performance.



Figure 3.2.2, Box plot of sales split based on promotion.

The box plot in Figure 3.2.2 shows the spread of sales across each promotion category. High-impact promotions have the widest range and the highest median sales, indicating their strong effect on revenue. Moderate-impact promotions show a more modest range and median, while low-impact promotions have the narrowest range and lowest median, highlighting their limited impact on sales.

In summary, both time and promotion intensity play a key role in sales performance. High-impact promotions around peak retail seasons maximise sales potential, while moderate-impact promotions maintain consistent sales. Low-impact promotions may have minimal influence on overall sales trends. This insight will guide our future promotional strategy, prioritising high-impact efforts during critical shopping periods.

Store Type/Assortment and Sales. Alongside exploring the impacts of promotion on sales, we emphasised the influence of assortment and storetype on mean sales through the use of a box plot and bar chart. The box plot was mainly used to showcase the distribution of the assortments sold by each storetype and plot it against the mean sales. The bar blot was to mainly show the total mean sales of each StoreType. This was to answer the question of whether or not the type of store affects sales. In order to plot these graphs, I utilised the `sns.boxplot()` method for the box plot and `plt.bar()` method for the bar plot.

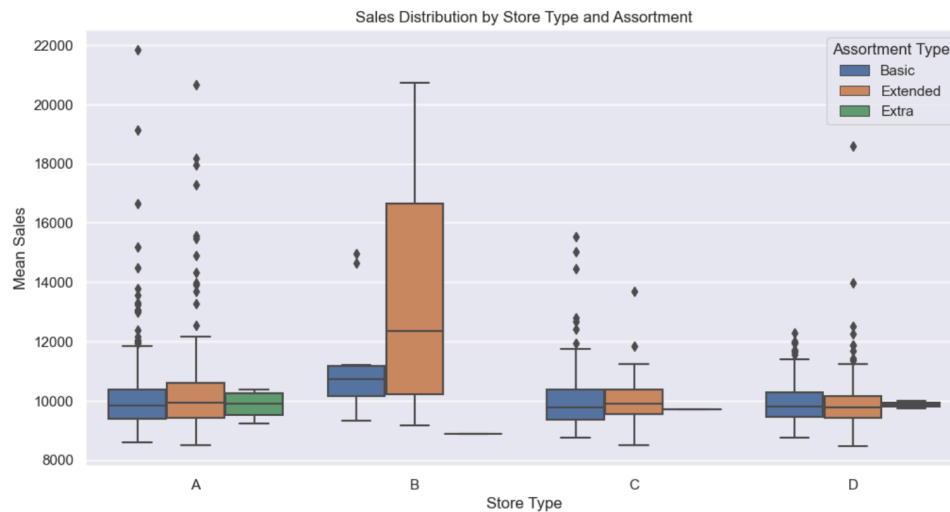


Figure 3.3.1, Box plot of sales for varying store type and assortments.

There are distinct patterns formed in figure 3.3.1 across the many stores.

StoreType A showcases the consistency in its mean sales across their assortments such that its median sales is relatively the same. Moreover, the variability of each assortment in store type A is relatively low, suggesting that the type of assortment sold primarily does not affect the Store type A's sales significantly. For an improvement, there are a few outliers in the Basic and Extended assortments that need to be dealt with and such removing them would have fixed the issue. In Contrast, **StoreType B** shows significant variability, especially with the extended assortment, suggesting that store type B has potential to achieve extremely high sales due to having only 17 stores, there is less overall stability in the distribution. **StoreType C** and **StoreType D** are on the same page when it comes to their impacts on mean sales as they exhibit lower and more stable sales, with limited impact from assortment type.



Figure 3.3.2, Bar chart of mean sales for different store types.

Figure 3.3.2 shows that StoreType **StoreType A** dominates the other store types when it comes to accumulating the greatest mean sales of all Storetype A's. Delving into the data, it was clear that StoreType A had the greatest number of stores and as such it is the main contributor to driving sales for Rossman. On the other hand **StoreType B**, despite having the greatest median sales, struggles to accumulate sales for rossman. This is a result of StoreType B having the lowest number of stores in rossman, which is also reflected in the variability of its box plots. **StoreType C** and **StoreType D** have lower median sales then its overshadowing counterpart (StoreType A) but still manages to have an influence on the total number of sales for Rossman.

The answer to the question of whether or not the type of store affects sales is that it does as a result of Store type A being the most successful in terms of both mean and consistent sales. Moreover, StoreTypes C and D also contribute significantly to having an influence on mean sales but not to the extent of store type A. Store Type B has almost a minimal effect on mean sales due to its lack of store data and such does not contribute too much to mean sales.

PREDICTIVE MODELLING

Multivariate Linear Regressions. The multivariate linear regression model was used to predict future Rossmann store sales based on various factors, such as promotions, competition distance,

store type, assortment, day of the week, and seasonality. This model was ideal for exploring the combined effect of these independent variables on sales, offering insights into their individual contributions as well as how they interact.

To prepare the data for regression, categorical variables such as store type and assortment were encoded into numerical formats using `sklearn.preprocessing`. The continuous variables, such as competition distance and day of the week, were included as is, allowing the model to capture their effect on sales variability.

To begin our analysis, we built a base regression model using the unprepared store data for both **Cumulative Sales** and **Average Sales** per store. The model performed slightly better when predicting **average sales** compared to **cumulative sales**, as shown in the accompanying figures. However, we anticipated that adding more features would significantly enhance the model's accuracy. (figure 4.1.1)

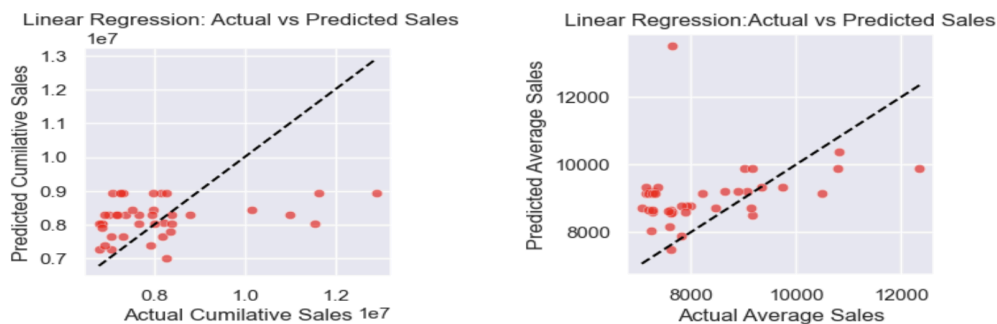


Figure 4.1.1, Initial regression model using unprepared data.

Mean Squared Error (MSE): 9827873.454
 Root Mean Squared Error (RMSE): 3134.944
 R-squared (R^2): 0.63



Mean Squared Error (MSE): 8137202.854
 Root Mean Squared Error (RMSE): 2852.578
 R-squared (R^2): 0.7

Figure 4.1.2, Error and R^2 scores for day-to-day sales and average sales.

While the base model gave us some insights, it became clear that additional features were necessary to capture more complex relationships within the data. In particular, when predicting **day-to-day sales**, the regression model performed decently, offering slightly better results than those for **cumulative or average sales** predictions by store or time. (figure 4.1.2)

We enhanced the model by merging various datasets and using **unique stores** as a key to integrate the different features. This process substantially improved the fit of the model, as indicated by reduced **Mean Squared Error (MSE)**, **Root Mean Squared Error (RMSE)**, and increased **R^2** scores. These improvements suggest that the enhanced model more effectively captures the relationships within the data, leading to better predictive performance.

After refining the initial model, we focused on utilising additional features while excluding the **customers** variable due to its near-perfect correlation with sales. Instead, we concentrated on factors such as **State Holidays**, **Promotions**, **Assortment Levels**. We also included features such as the month, day, distance and assortment types. We did this by using hot one encoding to change the different entries to binary results such as true or false.

We then further enhanced the model through ridge regularisation, we compared the results after using ridge and lasso regularisation and came to a conclusion that the ridge regularisation was a

better fit for the model. We used ridge regularisation because there were a large amount of feature variables and could have been multicollinearity, we also used it with intent to reduce overfitting and help control the complexity of the account.

This improved our model's performance significantly from the baseline. (figure 4.1.3)



Figure 4.1.3, Regressions after ridge regularisation based on different feature variables.

The improved model offered a better fit to the data, as demonstrated by the reduction in error metrics. By excluding the **customers** variable and leveraging more meaningful features, we were able to provide a more accurate prediction of day-to-day sales.

Overall, the refined model delivers stronger predictive power, demonstrating that features such as promotions, holidays, and assortment levels have a substantial impact on store sales.

Logistic Regressions. Logistic regressions predict whether a categorical variable is true or false, and as such we decided to form a logistic regression aimed at predicting whether a Rossmann store was a standard drug store (`store_type == 0`). Sales, competition distance, promo2 and assortment were used as the features in this model. One-hot encoding was used to split the assortment column into three new columns for each assortment type. Assortment had the greatest impact in this model since the magnitude of the regression coefficient of assortment type 2 was greater than triple that of sales, competition distance and promo2 (figure 4.2.1).

Sales: 0.038
 Competition distance: 0.237
 Ongoing promotion: 0.207
 Assortment type 1: 0.669
 Assortment type 2: -1.04
 Assortment type 3: 0.371

Figure 4.2.1, Regression coefficients for logistic regression predicting standard drug stores

This makes intuitive sense as the range of products sold should correlate highly with the type of store. Several different predictive thresholds were tested for this model and a threshold of 0.6 was found to yield the greatest accuracy of 0.675 and was therefore chosen for this model.

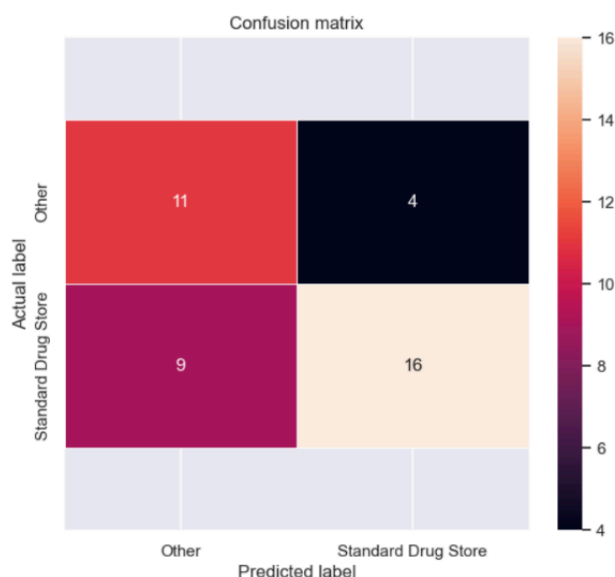


Figure 4.2.2, Confusion matrix for logistic regression predicting standard drug stores

The model had a high precision of 0.8, meaning that it was likely correct when it predicted a store to be a standard drug store. This statistic can be calculated directly from the confusion matrix (figure 4.2.2). This model showed that assortment type can be useful in predicting the type of store, but since the regression coefficient for sales was so low it did not reveal much about any relationship with sales. In order to find out more about sales, we split sales into a high

and low category based on whether a particular sales entry was highest or lower than the mean sales.

The goal of this new model is to determine how the presence or absence of promotions influences whether a store experiences high or low sales. Understanding the role promotions play in driving sales is critical for making informed business decisions, such as when to run promotions and which stores or product categories benefit most from them.

This logistic regression model predicts the probability of a store having high sales on a given day, based on a subset of features. By comparing models that include the 'Promo' feature with those that do not, we can quantify the effect of promotions and highlight their importance in forecasting sales.

```
sales_median = merged_df['Sales'].median()  
merged_df['HighSales'] = (merged_df['Sales'] > sales_median).astype(int)
```

In this case, we are using logistic regression to classify whether sales are high or low based on features like promotions, store type, competition distance, and other factors. For this regression model, we have created a binary target variable called 'HighSales'. This is a binary target variable we created, where 1 indicates that sales are above the median, and 0 indicates that sales are below the median. This allows us to classify each observation into either high or low sales categories.

The first logistic regression model we created will be used to classify whether sales will be high or low based on a subset of features. For this analysis, the following key features were included:

Promo: Indicates whether a store was running a promotion on a given day.

CompetitionDistance: The distance to the nearest competitor.

Promo2: A secondary type of promotion that some stores run.

Year and Month: Used to capture seasonality and time-based trends.


```
X = merged_df[['Promo', 'CompetitionDistance', 'Promo2', 'Year', 'Month']]
y = merged_df['HighSales']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
```

At first we called the `StandardScaler()` method to standardise the features by removing mean and scaling them. This is to help ensure that all features contribute equally to the model. Features with larger scales could otherwise dominate the learning process, as logistic regression is sensitive to the scale of input features. Standardisation brings all features to a comparable scale, with a mean of 0 and a standard deviation of 1. This ensures the model treats each feature equally during training

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

After calling the `StandardScaler` method, we call `scaler.fit_transform` on the `X_train` to scale the training data based on the scaler. Then, we use the `transform` method to apply the same scaling to the test data. This is important because standardisation ensures that our logistic regression model can handle the different scales of the features, allowing the model to learn more effectively and make better predictions.

```
log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train_scaled, y_train)
y_pred_log_reg = log_reg.predict(X_test_scaled)
```

Created an instance of logistic regression with `max_iter=1000`. This sets the maximum number of iterations the model can use to converge. By setting `max_iter` to 1000, we are ensuring the model has enough iterations to converge if necessary.

The `fit` method is used to train the logistic regression model, It takes the standardised training data and the target variable. After training the model, the `predict` method is used to make predictions on the test data. This method applies the learned model to the standardised test set

'X_test_scaled' to predict whether the sales on each day/store will be high (1) or low (0) - predictions are then stored in 'y_pred_log_reg'.

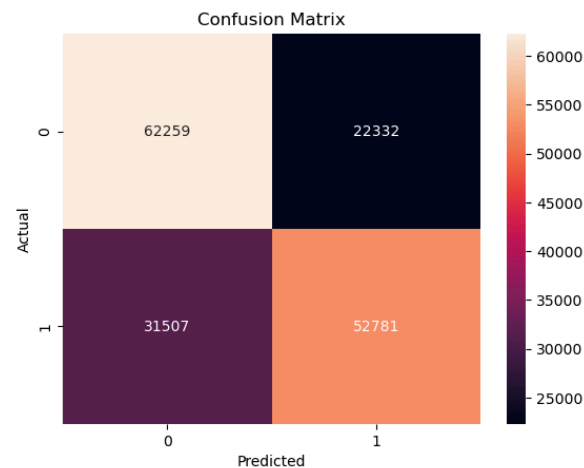


Figure 4.2.3, Confusion matrix for new logistic regression model with promo.

To represent the result of our model, we have created a confusion matrix (figure 4.2.3). It provides insight into the number of correct and incorrect predictions made by our model and helps describe its performance.

True Negatives (TN = 62,259): The model predicted low sales (class 0) and the actual sales were low.

False Positives (FP = 22,332): The model predicted high sales (class 1), but the actual sales were low.

False Negatives (FN = 31,507): The model predicted low sales (class 0), but the actual sales were high.

True Positives (TP = 52,781): The model predicted high sales (class 1) and the actual sales were high.

Classification Report:					
	precision	recall	f1-score	support	
0	0.66	0.74	0.70	84591	
1	0.70	0.63	0.66	84288	
accuracy			0.68	168879	
macro avg	0.68	0.68	0.68	168879	
weighted avg	0.68	0.68	0.68	168879	

Figure 4.2.4, Classification report for new logistic regression model with promo.

To summarise on the performance of our logistic regression model, we have created a classification report (figure 4.2.4) which consists of key metrics such as accuracy, precision, recall, as well as f1-score.

This logistic regression model with which includes promo as part of its feature variables is able to obtain an overall accuracy of 0.68. This value can be interpreted as how often the model correctly predicts whether sales will be high or low. It shows that the model is doing reasonably well at predicting sales with its 0.67 accuracy, though it is not perfect and shows that there is still room for improvement.

Our model achieved a precision score of 0.66 for class 0 (low sales). This means that 0.66 of the cases that are predicted to be low sales were actually low sales. On the other hand, it achieved a slightly better precision score of 0.70 for class 1 (high sales) which indicates that our model is better at predicting high sales than low sales. Overall, our model shows that it is slightly more reliable when it comes to predicting periods of high sales as a precision score of 0.70 shows that the model is somewhat confident in its high-sales predictions.

In terms of recall score, our model achieved a score of 0.74 and 0.63 for class 0 and class 1, respectively. This means that the model correctly identifies 0.74 of the actual low sales periods and 0.63 of the actual high sales periods. This disparity between recall score for class 0 and class

1 indicates that our model struggles more with recall for high sales compared to low sales, missing about one-third of high sales periods.

F1-score refers to the harmony or balance between precision and recall score for that particular class. We observe an f1-score of 0.70 and 0.66 for class 0 and class 1, respectively. This indicates that the model performs somewhat worse at predicting high sales than low sales. The f-1 score for class 1 is dragged down by its recall score, reflecting the fact that the model struggles to accurately predict periods of high sales.

The logistic regression model with promotions included performs reasonably well at predicting both high and low sales periods, with a precision of 0.70 for high sales and an overall accuracy of 0.68. This supports the hypothesis that promotions are a significant factor in determining sales outcomes, though there is room for improvement, particularly in increasing the recall for high-sales periods to capture more sales spikes.

To reiterate, the goal of this logistic regression model is to understand and determine how the presence or absence of promotions influences whether a store experiences high or low sales. Therefore, we conducted another logistic regression model but this time excluded 'Promo' as a feature variable to understand how our result would be impacted.

```
X = merged_df[['CompetitionDistance', 'Promo2', 'Year', 'Month']]
y = merged_df['HighSales']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
```

We repeat the same process of standardisation as well fitting our data to train the logistic regression model as we did before. Train-test-split is done the same way with test size of 0.2 as well as random state of 42. We have also constructed a confusion matrix to represent the findings of our model (figure 4.2.5).

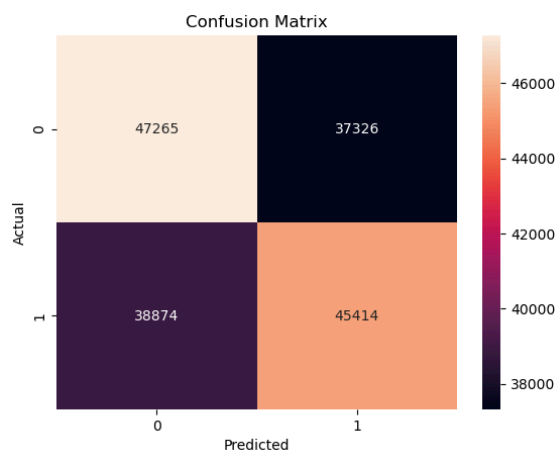


Figure 4.2.5, Confusion matrix for new logistic regression model without promo.

True Negatives (TN = 47,265): The model predicted low sales (class 0) and the actual sales were low.

False Positives (FP = 37,326): The model predicted high sales (class 1), but the actual sales were low.

False Negatives (FN = 38,874): The model predicted low sales (class 0), but the actual sales were high.

True Positives (TP = 45,414): The model predicted high sales (class 1) and the actual sales were high.

Observing our confusion matrix, it is clear to see that this model's performance pales in comparison with our previous model. This model recorded a high false positive as well as false negatives, indicating that its ability to predict high sales periods diminished greatly with 'Promo' excluded as part of the feature variable.

Classification Report:					
	precision	recall	f1-score	support	
0	0.55	0.56	0.55	84591	
1	0.55	0.54	0.54	84288	
accuracy			0.55	168879	
macro avg	0.55	0.55	0.55	168879	
weighted avg	0.55	0.55	0.55	168879	

Figure 4.2.6, Classification report for new logistic regression model without promo.

This classification report corresponding to our model (figure 4.2.6) reflects the idea that this model struggles to accurately make correct predictions. Looking at the accuracy score, it recorded a score of 0.55 - which is a 0.13 drop from our first mode. An accuracy of 0.55 means that the model only correctly predicts whether sales will be high or low 0.55 of the time. A drop of 0.13 implies that 'Promo' is a significant factor in predicting high sales periods.

A drop in precision score can also be observed. This model achieved a precision score of 0.55 both for class 0 as well as class 1. This score reflects a drop for both classes, 0.11 for class 0 and 0.15 for class 1. This result suggests that the model becomes more prone to making incorrect predictions, resulting in higher false negatives as well as false positives as shown previously.

Observing the recall score for both classes, we can see that it also experienced a drop in scores. For class 0, it experienced a drop of 0.18 from 0.74 to 0.56 whereas for class 1 it saw a drop from 0.63 to 0.54. Both indicate significant drops in score which further implies that this model struggles to capture high sales periods accurately.

The drop in both recall and precision scores is reflected on the f1-score which represents the balance between them. An f1-score of 0.55 and 0.54 were recorded for class 0 and class 1, respectively. This again confirms that the model struggles to predict accurately periods of high sales with the absence of the 'Promo' variable.

Summary Table			
Metric	With 'Promo'	Without 'Promo'	Change
Precision class 0	0.66	0.55	-0.11
Precision class 1	0.70	0.55	-0.15
Recall class 0	0.74	0.56	-0.18
Recall class 1	0.63	0.54	-0.09
F1-score class 0	0.70	0.55	-0.15
F1-score class 1	0.66	0.54	-0.12
Accuracy	0.68	0.55	-0.13

Figure 4.2.7, Table summarising the two classification reports for new logistic regressions

The comparison of the two classification reports (figure 4.2.7) highlights the critical role 'Promo' plays in predicting sales outcomes. In conclusion, the logistic regression analysis strongly supports the idea that promotions are essential for driving high sales and for making accurate predictions about store performances.

Random forest regressor. Random forest regressor is a regression that predicts a target variable by averaging the outputs of multiple decision trees, aiming to capture different aspects of data to find an accurate mean value. This was a perfect regressor to use in the cases of discovering how multiple events had affected the average sales of Rossmann's store day by day activity. In the creation of the regressor, data used were whether that day of sale was a state holiday, school holiday, promo, competition distance, store type, assortment, day of the week and date (which was used to analyse month and season). As the regressor only split its nodes through integers, categorical values were transformed into categorical values using library sklearn.preprocessing.

Through the analysis of feature importance (figure 4.3.1), it was found that competition distance had an overwhelming effect on sales, having a importance score of 0.55, which was then followed by promotions with a feature of importance score of 0.20. Factors such as day of week, month, store type and assortment has a minor effect having a feature importance score score of around 0.1, while factors such as season state holiday and school holidays had little to no effect on sales, having order of importance score less that 0.1

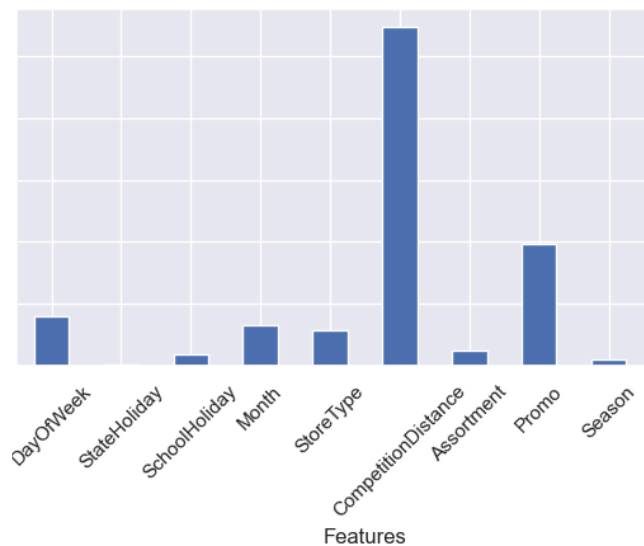


Figure 4.3.1, Bar graph of feature importance in random forest regressor model.

When I came to fine tuning the configurations, creating hyper parameters, which tested all many parameters to find the most optimal set of values that would lead to the highest accuracy. I was determined that using sklearn's default settings was the best, which was:

`n_estimators : 100`

`max_depth: None`

`min_samples_split: 2`

`min_sample_leaf: 1`

After the fine tuning, it can be found that the random forest regressor generated lead to a high accuracy (figure 4.3.2). Having a r^2 value of 0.7 means that the model explains 70% of the variance in data. Having a MAE of 938.80 shows the difference between prediction in sales and actual values is on average 938.90 euros. Overall with these figures, the model created had a high accuracy and was fitted fairly well.

Mean Squared Error: 1789309
Mean Absolute Error: 938.80
R² Score: 0.70

Figure 4.3.2, Error and R² score for random forest regressor model.

Through figure 4.3.3, it can be seen that the first split is promotions, indicating that when there is a promotion active, average sales tend to be above €6586.80, while without it would be below the given value. To the left subtree, it can be seen that having a basic assortment will lead to a lower average sales value of €5727.13, while having extra or extended tends to lead to higher average sales. At the end of the left subtree, it can be interpreted that being further away from competition will lead to an increase in average sales, being from 26.5km leads to an average sales of €5418.00 and when a store is set up further away it will increase average sales up to €6074.26 at 33.5km, and is estimated to increase the further it get away from competition. To the right it can be seen that when there are promotions, days of week and month usually play an effect on average sales. As seen in the first layer, Sunday will have fewer average sales while every other day will lead to higher average sales. This information is slightly distorted due to having most Rossmann stores closed on Sunday. The next layer will then look at months, where it found that average sales in December will lead to higher average sales, while other average sales seem to be lower.

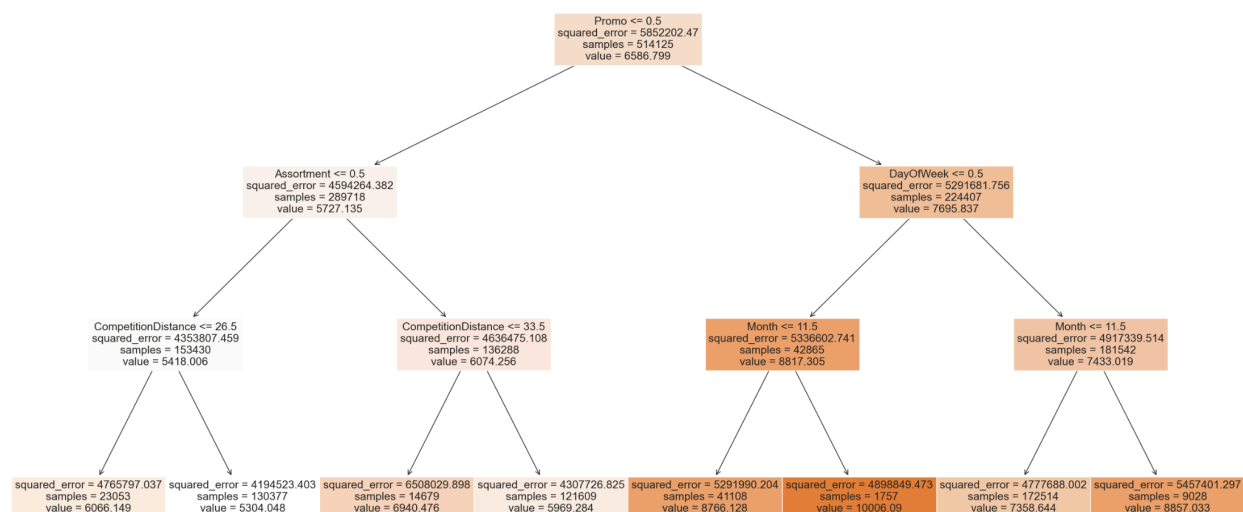


Figure 4.3.3, Decision tree from random forest regressor model.

Random Forest Classifiers. Like the random forest regressor, the random forest classifier utilises multiple decision trees that are formed based on different features and data, and are trained to to predict target variables..

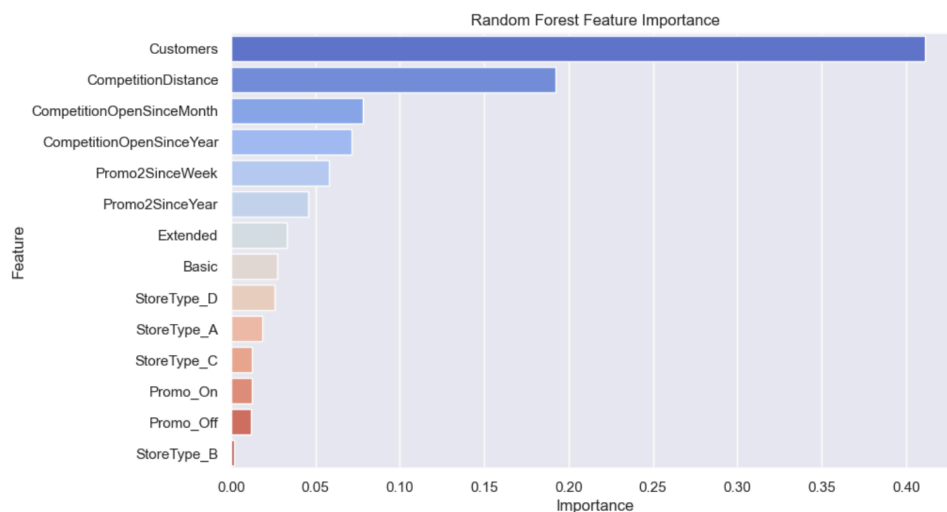


Figure 4.4.1, Importance of features in random forest classifier.

Moreover, whichever label is predicted the most frequently by the tree is used for the final prediction. However, classifiers predict the labels for given inputs unlike a regressor that predicts the actual values for imputed features. During data collection, one-hot encoding was used to convert categorical variables such as `Promo_On` and `Promo_Off` into numerical variables such as 0s and 1s. Furthermore, the Test data was used to see how well the model performed. We also hyper parameterised the model in order to find the best accuracy, precision and recall for our model which can be seen in Figure 4.4.2.

```
random_state=42, n_estimators=100, max_depth = 20, min_samples_leaf = 4, min_samples_split = 10)
```

Figure 4.4.2, Hyper parameterisation of random forest classifier.

Figure 4.4.2 shows **random state** of 42 which ensures the model is trained on the same data, **n_estimators** of 100 which uses 100 trees to improve model efficiency, **max_depth** of 20 which allows each of the 100 trees to split up to 20 levels deep and reduce overfitting, **min_samples_leaf** of 4 highlights the minimum number of samples that needs to be present in the leaf node to allow for more general decisions, and **min_samples_split** of 10 which allows for the model to split when there are at least 10 samples, minimising overfitting.

Customers have the greatest importance to predicting high versus low sales (with median being the pivot), suggesting that if customers were to be omitted during inputting store values, it would force the model to predict inaccurate results.

Competition distance/CompetitionOpenSinceMonth/CompetitionOpenSinceYear/Promo2SinceWeek, PromoSinceYear were all important to predicting high versus low sales alluding to competition distance, promotions and competition all have an influence on differentiating between high and low sales

StoreTypes, Assortment and Promo_on/Promo_Off: All were less significant to the model but still contributed to accurate predictions.

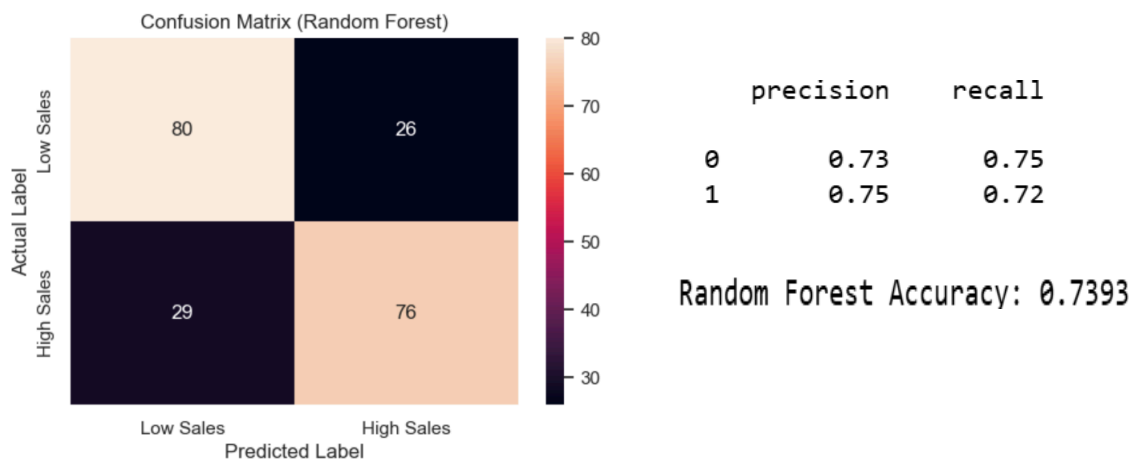


Figure 4.4.3, Confusion matrix, precision, accuracy and recall of Random Forest Classifier.

Figure 0 corresponds to Low sales, 1 corresponds to High Sales

The **accuracy** of 73.93% suggests that the model is reliable for predictions of labels such that it is in the range of having no overfitting or underfitting in the model. However, there are some misclassifications, suggesting that accuracy can still be improved. This could be potentially done by reducing noise in the data set and removing more outliers.

The **precision** of 75% for high sales and 72% for low sales suggests that the model is slightly better at correctly predicting High sales compared to low sales. However, the precision itself was strong but could be improved by using more important features.

The **recall** of 72% for high sales and 75% for low sales suggests that the model fails to predict high sales 28% of the time and low sales 25% of the time, further suggesting that high sales predicts false negatives worse than low sales. Overall, the model misses some opportunities to correctly identify high sales when compared to low sales.

Overall, the random forest regression model demonstrates reliable predictive performance as a result of customers and competition distance driving accuracy. However, improvements such as enhancing the precision and recall could benefit the model and gain more comprehensive predictions, potentially by using oversampling techniques such as SMOTE (Synthetic Minority Oversampling Technique).

Support Vector Machines. Support Vector Machines (SVM) is a supervised machine learning algorithm that is primarily used for classifying targets (ie. High vs Low sales). It operates by finding the most optimal hyperplane by separating the target labels. It does this by maximising the margin between support vectors and the hyperplane to ensure the data points are being classified as accurately as possible. In our case, a kernel trick (kernel = 'rbf') was required to deal with non-linear data (store type), minimising noise when finding the optimal hyperplane. The RBF kernel was chosen as a result of its flexibility in handling non-linearly separable data such that it allows for intricate patterns to be captured. Moreover, Support vector machines were mainly used for the purpose of highlighting if outliers had a major impact on models, due to SVMs being sensitive to outliers. Lastly, we used the median as the pivot such that any data above it are classified as high sales and any data below it are classified as low sales

(For underlined words refer to **APPENDIX**)

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

Figure 4.5.1, Code for Standardising

I utilised feature standardisation by using StandardScaler() to ensure that all features have a mean of 0 and standard deviation of 1, preventing noise from skewing the boundaries and decreasing overall model accuracy.

```
svm_clf = SVC(kernel='rbf', random state=42)
```

Figure 4.5.2, Code for using a kernel

Moreover, I used the RBF kernel to allow for the model to handle non-linear data relationships, improving accuracy, precision and recall of the model by preventing noise from influencing the hyperplane.

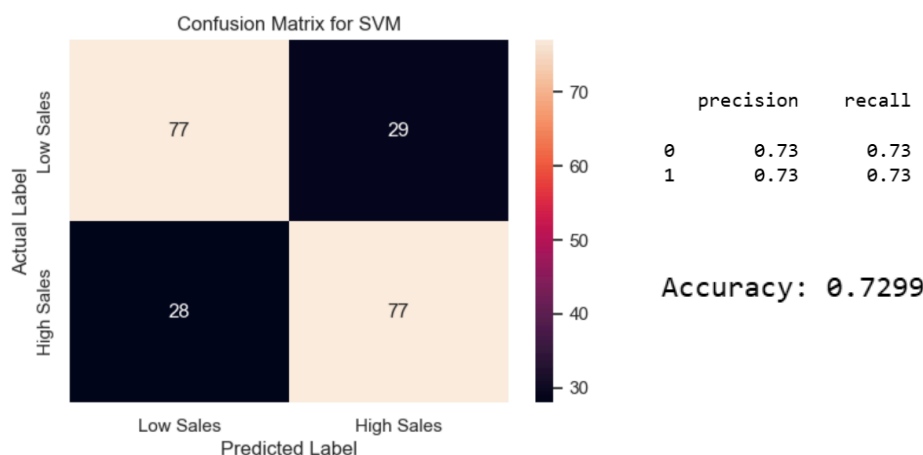


Figure 4.5.3, The confusion matrix, accuracy, precision and recall. **0 corresponds to Low sales, 1 corresponds to High Sales.**

The SVM achieved an **accuracy** of 73%, meaning that 73% of predictions made by the model were correct, indicating that the model is reliable at distinguishing between high and low sales. Moreover, the accuracy suggests that the model is not being overfitted nor under-fitted such that it is not memorising data patterns nor randomly guessing predictions. Despite the great accuracy, there is still room for improvement such that 27% of the model's predictions were incorrect, which is greater than the inaccuracies of the random forest regression, alluding to the presence of some outliers that may be influencing the hyperplane of that model.

The SVM achieved a **precision** and **recall** of 0.73 for low and high sales, suggesting that the model has a balanced performance overall. The precision for predicting high sales means that 73% of the predictions labelled as high sales were correct and 73% of the predictions labelled as low sales were correct, suggesting the model does not have too many false positives. Moreover, the recall for high and low sales was 0.73, meaning the model correctly identified 73% of high sales and low sales instances. However, both high and low sales had 27% of their predictions become misclassified. The balanced precision and recall results suggest that there is no substantial bias towards either high or low sales whilst forming predictions, thus making it a fitting model for determining whether sales are high or low.

Overall, the hyperplane is not influenced by outliers too significantly as the model's accuracy is not too far away from the accuracy of models such as random forest. However, the SVM accuracy was still less than that of the random forest classifier, suggesting that outliers may still skew the boundaries of the SVM. To further improve the accuracy of the model, outliers could be identified and removed from the data set. Furthermore, the model has a strong overall accuracy and its precision and recall are ideally balanced, thus being a strong model for forming predictions.

CONCLUSIONS

After some brief data wrangling, we conducted exploratory data analysis on several variables in the store and train dataframes provided. It was found that variables such as competition distance and promo all had a noticeable effect on the sales of a Rossmann store, and so were all included in our predictive models.

Our most accurate model was the random forest classifier which had an accuracy of 73.93%, likely due to the fact it used the customers variable. The customers variable had clearly the greatest influence on the model and since this correlation between sales and customers is obviously expected, other predictive models excluded customers. Even though customers makes predicting sales easier, we were still able to produce a logistic regression that predicted high/low

sales with a decent accuracy of 68.12% which demonstrated that variables other than customers still have a great enough impact on sales to allow for reliable predictions to be drawn.

Although varying conclusions were drawn from different models, it was generally found that besides customers, competition distance and promo were of the greatest importance when predicting sales. Store type, assortment and holidays were also relevant.

If Rossmann wishes to maximise sales in the future, they should increase the number of ongoing promotions in their stores. Also, stores should ideally be set up in an area with high foot traffic even if it means being within walking distance of the nearest competing store. However, it should be kept in mind that being too close to the nearest competitor will have detrimental effects on sales. Certain times of the week and year generate a larger number of sales and therefore Rossmann should aim to increase their opening hours during Mondays and Sundays and number of days they are open during December. It should also be noted that Rossmann's most successful stores were their convenience stores that offered an extended assortment of items that went beyond their standard range, so Rossmann should aim to open as many of these types of stores as possible.

For future analysis, it is recommended that Rossmann stores provide data relating to a store's geographical location and the foot traffic in that area. This would greatly help in analysing the effect of distance to nearest competition on sales and also make room for new analysis that can explore the effect of a store's location on its sales. Interesting findings such as whether or not high performing stores are located near each other or which countries Rossmann stores make the greatest number of sales in could be drawn from this kind of data. Additionally, some data such as the year since the nearest competition was introduced was not complete and so was difficult to find a use for in our analysis. Filling in this data and making it more complete would assist future projects in finding more meaningful conclusions.

REFERENCES

Deepchecks. (2022, October 18). *Hyperplane in Machine Learning*. Deepchecks

<https://www.deepchecks.com/glossary/hyperplane/>

Hosni, Y. (n.d.). *Practical Guide to Support Vector Machines in Python*. GitHub

<https://github.com/youssefHosni/Practical-Machine-Learning/blob/main/Practical%20Guide%20to%20Support%20Vector%20Machines%20in%20Python%20.ipynb>

Scikit-learn. (n.d.). *Support Vector Machines*. Scikit-learn.

<https://scikit-learn.org/1.5/modules/svm.html>

APPENDIX

Support Vector Machines

Support Vector Machines (SVM) are a set of supervised machine learning methods used for classification, regression and outlier detection (Scikit-learn, n.d.). The main intention of SVM is to find the best hyperplane that separates data points from classifications.

List of the key terminologies:

Hyperplane:

Hyperplane is a decision boundary that divides the input space into two or more regions, each corresponding to a different class or output label. In a 2D space, a hyperplane is a straight line that divides the space into two halves. In a 3D space, however, a hyperplane is a plane that divides the space into two halves. Meanwhile in higher-dimensional spaces, a hyperplane is a subspace of one dimension less than the input space. (Deepchecks, 2022)

Margin

Margin is the distance between the hyperplane and the nearest data points from each classification (High versus Low sales)

Support vector

Support vectors are data points that lie closest to the hyperplane and are what influences the hyperplanes orientation.

Kernel trick

A method that transforms data into a higher dimension (used kernel = 'rbf') in order to find an optimal hyperplane that is not linearly separable, such that it reduces noise from data and does not skew the hyperplane

Code inspiration

```
from sklearn.svm import SVC
```

```
X, y = make_moons(noise=0.3, random_state=0)
```

```
xx, yy = np.meshgrid(np.linspace(-3, 3, 500), np.linspace(-3, 3, 500))
```

```
np.random.seed(0)
```

```
rbf_kernel_svm_clf = Pipeline([
```

```
    ("scaler", StandardScaler()),
```

```
    ("svm_clf", SVC(kernel="rbf", gamma=5, C=1000))
```

```
])
```

```
rbf_kernel_svm_clf.fit(X, y)
```

```
decision_boundary_plotting(xx, yy, rbf_kernel_svm_clf, 'SVM classifier using RBF Kernel features  $\gamma=5$ ,  $C=1000$ ')
```

(Hosni, n.d.)